Dataframe

Data Structures

- vectors
- factors
- lists
- matrices
- arrays
- ***`dataframes`***

Dataframe

The dataframe is the most “complex”, useful and powerful structure in R. Intuitively it is an excel sheet, while from R’s point of view it is a type of **list** with some properties/restrictions

- each element of the list is a **vector** with an **associated name** (aka a column)
- each list/column must have the same number of elements
- as a consequence, each row has the same number of elements (**rectangular structure**)

Creation

Most of the time you will import dataframes rather than create them, but it is important to first understand how they work and how to create/manipulate them. They are created with the `data.frame` command

```
# Create a dataframe with 3 columns
my_df <- data.frame( col1 = 1:4, col2 = letters[1:4],
                     col3 = rnorm(n = 4, mean = 0, sd = 1))
my_df
```

```
  col1 col2       col3
1    1    a  1.7779579
2    2    b -0.2343056
3    3    c  1.5753641
4    4    d  2.0025113
```

Attributes

The **dataframe** has both the attributes of a list, i.e. the ***names***, and the attributes of a matrix, i.e. the ***dimensions*** (rows and columns)

```
1 attributes(my_df)
```

```
$names
[1] "col1" "col2" "col3"

$class
[1] "data.frame"

$row.names
[1] 1 2 3 4
```

```
1 dim(my_df)
```

```
[1] 4 3
```

As with the other structures we have seen, we can use the functions **`names()`** , **`dim()`**, **`nrow()`**, **`ncol()`**… to get information about the properties of the dataframe.

```r
names(iris) # iris is a dataset available by default in R
```

```
[1] "Sepal.Length" "Sepal.Width"  "Petal.Length"
"Petal.Width"  "Species"
```

```r
nrow(iris)
```

```
[1] 150
```

```r
ncol(iris)
```

```
[1] 5
```

The most useful function is **`str()`**, since it gives us a quick overview of the structure of the dataframe: dimensions, variable types,…

```
1 str(iris)
```

```
'data.frame':   150 obs. of  5 variables:
 $ Sepal.Length: num  5.1 4.9 4.7 4.6 5 5.4 4.6 5 4.4 4.9 ...
 $ Sepal.Width : num  3.5 3 3.2 3.1 3.6 3.9 3.4 3.4 2.9 3.1
...
 $ Petal.Length: num  1.4 1.4 1.3 1.5 1.4 1.7 1.4 1.5 1.4 1.5
...
 $ Petal.Width : num  0.2 0.2 0.2 0.2 0.2 0.4 0.3 0.2 0.2 0.1
...
 $ Species     : Factor w/ 3 levels "setosa","versicolor",..:
1 1 1 1 1 1 1 1 1 1 ...
```

Indexing

We can use either square brackets `[]`:

```
1 my_df
```

```
  col1 col2       col3
1    1    a  1.7779579
2    2    b -0.2343056
3    3    c  1.5753641
4    4    d  2.0025113
```

```
1 my_df[1]  # extract a 4x1 data.frame
```

```
  col1
1    1
2    2
3    3
4    4
```

```r
my_df[[1]] # extract the first column of the data.frame
```

```
[1] 1 2 3 4
```

```r
my_df[1,1] # extract the first element of the first column
```

```
[1] 1
```

…or the dollar sign `$`

```
# my_df[[1]] equivalent
my_df$col1 # extract the first column of the data.frame
```

```
[1] 1 2 3 4
```

… extract the first element of the first column…

```
my_df$col1[1]
```

```
[1] 1
```

Indexing - Relational operators

One of the most common operations you will deal with is extracting/evaluating a subset of the values in your dataset:

```
1 my_df
```

```
  col1 col2       col3
1    1    a  1.7779579
2    2    b -0.2343056
3    3    c  1.5753641
4    4    d  2.0025113
```

```
1 # keep only the rows where the values of column1 are great
2 my_df[my_df$col1 > 2, ] # = my_df[my_df[1] > 2, ]
```

```
  col1 col2     col3
3    3    c 1.575364
4    4    d 2.002511
```

Examples

```
  col1 col2       col3
1    1    a  1.7779579
2    2    b -0.2343056
3    3    c  1.5753641
4    4    d  2.0025113
```

```r
my_df[my_df$col1 > 2 & my_df$col1 < 4, ]
```

```
  col1 col2     col3
3    3    c 1.575364
```

```r
my_df[my_df$col1== 2,  2]
```

```
[1] "b"
```

```r
my_df[my_df$col1== 2,  "col2"]
```

```
[1] "b"
```

Indexing

There are also alternative, more compact ways of indexing. For example, using the `subset()` function:

```
1 subset(iris, subset = Species == "setosa" & Petal.Length >
```

```
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
25          4.8         3.4          1.9         0.2  setosa
45          5.1         3.8          1.9         0.4  setosa
```

equivalent to:

```
1 iris[iris$Species == "setosa" & iris$Petal.Length > 1.7,]
```

```
   Sepal.Length Sepal.Width Petal.Length Petal.Width Species
25          4.8         3.4          1.9         0.2  setosa
45          5.1         3.8          1.9         0.4  setosa
```

We can also select columns rather than rows with the ***select*** argument:

```
1 subset(iris, select = c(Sepal.Length, Species))
```

```
    Sepal.Length    Species
1            5.1     setosa
2            4.9     setosa
3            4.7     setosa
4            4.6     setosa
5            5.0     setosa
6            5.4     setosa
7            4.6     setosa
8            5.0     setosa
9            4.4     setosa
```

We can also combine the two:

```
subset(iris, subset = Species == "setosa" & Sepal.Length >
       select = c(Sepal.Length, Species))
```

```
  Sepal.Length Species
1          5.1  setosa
2          4.9  setosa
3          4.7  setosa
4          4.6  setosa
5          5.0  setosa
```

... equivalent to:

```
iris[iris$Species == "setosa" & iris$Sepal.Length > 4,
     c("Sepal.Length","Species")]
```

Indexing

Most of the time you will access variables with the `$` operator. This command can also be used to create a new variable…

```
# create a variable that is the sum of Length and Width
iris$sum <- iris$Sepal.Length + iris$Sepal.Width
iris[, c("Sepal.Length", "Sepal.Width", "sum")]
```

```
    Sepal.Length Sepal.Width  sum
1            5.1         3.5  8.6
2            4.9         3.0  7.9
3            4.7         3.2  7.9
4            4.6         3.1  7.7
5            5.0         3.6  8.6
6            5.4         3.9  9.3
```

The same concepts we have seen for vectors apply, so you can both create and modify variables…

```r
my_df <- data.frame(num = 1:4, let = letters[1:4])

# Modify the num variable by adding 1
my_df$num <- my_df$num+1
my_df$num
```

```
[1] 2 3 4 5
```

```r
# Create a third variable ?paste
my_df$both <- paste(my_df$num, my_df$let, sep = "_")
my_df$both
```

```
[1] "2_a" "3_b" "4_c" "5_d"
```

Combining Dataframes

Dataframes can be combined with the **`rbind()`** function:

```r
# create a second dataframe
my_df2 <- data.frame(num = 4:7, lett = letters[1:4],
                     both = paste(4:7,letters[1:4], sep = "

# join the two
my_df3 <- rbind(my_df,my_df2)
```

```
Error in `match.names()`:
! names do not match previous names
```

Rules

- The dataframes must have the same number of columns
- The column names must be identical

```
1 str(my_df)
```

```
'data.frame':   4 obs. of  3 variables:
 $ num : num  2 3 4 5
 $ let : chr  "a" "b" "c" "d"
 $ both: chr  "2_a" "3_b" "4_c" "5_d"
```

```
1 str(my_df2)
```

```
'data.frame':   4 obs. of  3 variables:
 $ num : int  4 5 6 7
 $ lett: chr  "a" "b" "c" "d"
 $ both: chr  "4_a" "5_b" "6_c" "7_d"
```

Fix the names

```
1 names(my_df2)
```

```
[1] "num"  "lett" "both"
```

```
1 names(my_df)
```

```
[1] "num"  "let"  "both"
```

```
1 # I want the names of my_df2 to match the names of my_df
2 names(my_df2) <- names(my_df)
3 
4 my_df3 <- rbind(my_df,my_df2)
5 str(my_df3)
```

```
'data.frame':   8 obs. of  3 variables:
 $ num : num  2 3 4 5 4 5 6 7
 $ let : chr  "a" "b" "c" "d" ...
 $ both: chr  "2_a" "3_b" "4_c" "5_d" ...
```

You may also need to collect different types of data from the same participant, and then combine the collected information…

```
df_rt <- data.frame(subj = factor(rep(c("alice","bob"), ea
                    cond = factor(rep(c("easy","hard"),
                                      each = 30, times = 2)
                      rt = c(rlnorm(n = 30, meanlog = -1,
                             rlnorm(n = 30, meanlog = -.7,
                             rlnorm(n = 30, meanlog = -1,
                             rlnorm(n = 30, meanlog = -.7,
df_rt
```

```
    subj cond        rt
1  alice easy 0.2156427
2  alice easy 0.4541752
3  alice easy 0.2601258
4  alice easy 0.2454218
5  alice easy 0.3972711
6  alice easy 0.2506634
```

```
7   alice easy 0.4182994
8   alice easy 0.2366576
9   alice easy 0.3122877
```

Dataframe containing the age:

```
df_age <- data.frame(subj = factor(c("alice","bob")), age
df_age
```

```
   subj age
1 alice  20
2   bob  18
```

```r
# structure of the reaction times dataframe
str(df_rt)
```

```
'data.frame':   120 obs. of  3 variables:
 $ subj: Factor w/ 2 levels "alice","bob": 1 1 1 1 1 1 1 1 1
1 ...
 $ cond: Factor w/ 2 levels "easy","hard": 1 1 1 1 1 1 1 1 1
1 ...
 $ rt  : num  0.216 0.454 0.26 0.245 0.397 ...
```

```r
# structure of the age dataframe
str(df_age)
```

```
'data.frame':   2 obs. of  2 variables:
 $ subj: Factor w/ 2 levels "alice","bob": 1 2
 $ age : num  20 18
```

In this case, we can use the `merge()` function …

```
df_all_1 <- merge(x = df_rt, y = df_age, by="subj")
df_all_1
```

```
    subj cond        rt age
1  alice easy 0.2156427  20
2  alice easy 0.4541752  20
3  alice easy 0.2601258  20
4  alice easy 0.2454218  20
5  alice easy 0.3972711  20
6  alice easy 0.2506634  20
7  alice easy 0.4182994  20
8  alice easy 0.2366576  20
9  alice easy 0.3122877  20
```

… or the `_join()` function..

```r
library(dplyr) # load the dplyr package

df_all_2 <- left_join(x = df_rt, y = df_age, by = c("subj"
df_all_2
```

```
    subj cond        rt age
1  alice easy 0.2156427  20
2  alice easy 0.4541752  20
3  alice easy 0.2601258  20
4  alice easy 0.2454218  20
5  alice easy 0.3972711  20
6  alice easy 0.2506634  20
7  alice easy 0.4182994  20
8  alice easy 0.2366576  20
9  alice easy 0.3122877  20
```

```r
df_rt$gender <- rep(c("F","M"), each = 60)
df_age$gender <- c("F","M")

df_all_g <- merge(x = df_rt, y = df_age, by=c("subj","gend
df_all_g
```

```
    subj gender cond        rt age
1  alice      F easy 0.2156427  20
2  alice      F easy 0.4541752  20
3  alice      F easy 0.2601258  20
4  alice      F easy 0.2454218  20
5  alice      F easy 0.3972711  20
6  alice      F easy 0.2506634  20
7  alice      F easy 0.4182994  20
8  alice      F easy 0.2366576  20
9  alice      F easy 0.3122877  20
```

Contingency tables

We can use the `table()` function both to compute frequencies…

```r
# keep only rts between .250 and 1.5 seconds
# i.e. the rows that meet this condition
df_rt_clean <- df_rt[df_rt$rt > .25 & df_rt$rt < 1.5, ]
table(df_rt_clean$subj) # trials for each subject
```

```
alice   bob
   57    57
```

```r
table(df_rt_clean$subj, df_rt_clean$cond) # subject x cond
```

```
       easy hard
  alice   27   30
  bob     27   30
```

Exporting and importing data

In R we can import data in many different formats; most commonly you will import ***.csv*** or ***.xlsx*** data.

Here, for example, I export the dataframes in two different formats…

```
library(readr) # load the readr package
library(writexl) # load the writexl package

write.csv(df_rt, file = "DATA/df_rt.csv",row.names = FALSE
write_xlsx(df_age, path = "DATA/df_age.xlsx")
```

Importing

```r
library(readxl) # load the readxl package

df_rt_impo <- read_csv("DATA/df_rt.csv") # uses the readr

df_age_impo <- read_xlsx("DATA/df_age.xlsx")
```

Checking the structure

```r
str(df_rt) # Data frame we created
```

```
'data.frame':   120 obs. of  4 variables:
 $ subj  : Factor w/ 2 levels "alice","bob": 1 1 1 1 1
1 1 1 1 1 ...
 $ cond  : Factor w/ 2 levels "easy","hard": 1 1 1 1 1
1 1 1 1 1 ...
```

```r
str(df_rt_impo) # Data frame we loaded
```

```
spc_tbl_ [120 × 4] (S3:
spec_tbl_df/tbl_df/tbl/data.frame)
 $ subj  : chr [1:120] "alice" "alice" "alice" "alice"
...
 $ cond  : chr [1:120] "easy" "easy" "easy" "easy" ...
```

The dataframe is imported as a tibble, and in this case the ***factor*** variables have been imported as ***character***. This can be changed either when importing (**?read_csv**):

```
df_rt_impo1 <- read_csv("DATA/df_rt.csv",
                        col_types =
                          list(col_factor(),col_factor(),
                               col_double()))
str(df_rt_impo1)
```

```
spc_tbl_ [120 × 4] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ subj  : Factor w/ 2 levels "alice","bob": 1 1 1 1 1 1 1 1
1 1 ...
 $ cond  : Factor w/ 2 levels "easy","hard": 1 1 1 1 1 1 1 1
1 1 ...
 $ rt    : num [1:120] 0.216 0.454 0.26 0.245 0.397 ...
 $ gender: chr [1:120] "F" "F" "F" "F" ...
 - attr(*, "spec")=
  .. cols(
```

```
..   subj = col_factor(levels = NULL, ordered = FALSE,
```

or afterwards with the `as.factor()` command:

```
df_rt_impo$subj <- as.factor(df_rt_impo$subj)
df_rt_impo$cond <- as.factor(df_rt_impo$cond)
str(df_rt_impo)
```

```
spc_tbl_ [120 × 4] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ subj  : Factor w/ 2 levels "alice","bob": 1 1 1 1 1 1 1 1
1 1 ...
 $ cond  : Factor w/ 2 levels "easy","hard": 1 1 1 1 1 1 1 1
1 1 ...
 $ rt    : num [1:120] 0.216 0.454 0.26 0.245 0.397 ...
 $ gender: chr [1:120] "F" "F" "F" "F" ...
 - attr(*, "spec")=
  .. cols(
  ..   subj = col_character(),
```

equivalent to:

```r
df_rt_impo2 <- read_csv("DATA/df_rt.csv") %>% #library(tid
  mutate(subj = factor(subj),
         cond = factor(cond))
str(df_rt_impo2)
```

```
tibble [120 × 4] (S3: tbl_df/tbl/data.frame)
 $ subj  : Factor w/ 2 levels "alice","bob": 1 1 1 1 1 1 1 1
1 1 ...
 $ cond  : Factor w/ 2 levels "easy","hard": 1 1 1 1 1 1 1 1
1 1 ...
 $ rt    : num [1:120] 0.216 0.454 0.26 0.245 0.397 ...
 $ gender: chr [1:120] "F" "F" "F" "F" ...
```

(just to show you that in ***R*** a problem can be solved in different ways)

The examples I have shown concern data in formats **external** to R, but we can also save and import data in `R` formats.

Their main drawback is that they can be read (mainly) only within the R environment.

If we need to work with different software or languages, it is probably better to use a more generic format (e.g, .csv or .txt).

Now let’s practice!

Open this link and keep it open:

https://etherpad.wikimedia.org/p/arca-corsoR